Formula 1 Podium Prediction

CSEN903 Advanced Computer Lab — Milestone 1 Report

Team members

Abdelhamid Sameh (58-1165)
Mohamed Ayman (58-8189)
Noureldeen Amr (58-1632)
Noureldin Abdelrahman (58-7007)

October 2026

Notebook: `notebooks/f1_podium_prediction.ipynb`
Kaggle: `https://www.kaggle.com/code/nooredeenamr/notebook03d3448a5a`
GitHub: `https://github.com/MohamedAyman04/f1-drivers`

Abstract

We predict, for every driver in every Formula 1 race from 1950 to 2024, whether the driver finishes on the podium (`positionOrder` ≤ 3), using only what is known after qualifying and before the start. The 14 raw tables are audited and cleaned on copies (160,076 `\N` sentinels, 85 duplicated driver-race pairs, all keys checked), the three data-engineering questions are answered with multi-hop joins, and a leakage-safe modeling table of 24,933 entries is built and split by season. Feature groups are added one at a time on the validation seasons; only the starting grid and the driver's championship standings before the race earn their place. A logistic regression, a random forest and a shallow neural network all beat a grid-only baseline: test PR-AUC 0.708, 0.720 and 0.706 against 0.647, with no model clearly ahead of the others. SHAP and LIME show that every model relies mostly on the grid. These explanations describe the models, not what causes podiums. Section numbers in the form *(notebook §9.1)* point to the matching section of the notebook, where every number in this report is printed.

Contents

1 Task and dataset **3**

2 Exploratory data analysis before cleaning **3**
2.1 First look and sentinels . . . 3
2.2 Where and when values are missing . . . 3
2.3 Spelling, grid slots and field sizes . . . 4
2.4 Keys and duplicates . . . 5

3 Cleaning **5**

4 Target, pre-race cutoff and metrics **6**

5 Data-engineering questions **7**
5.1 Q1: Which circuits convert a front-row start into a podium most often, and did that change after 2014? . . . 7
5.2 Q2: Do drivers podium more often at home, for the same starting position? . . . 8
5.3 Q3: Which constructors have the highest mechanical-retirement rate in 2014–2021 vs. 2022–2024? . . . 9

6 Modeling table and data split **10**

7 Feature engineering and selection **11**
7.1 Candidate features and why they are valid at the cutoff . . . 11
7.2 How many races should recent form look back? . . . 11
7.3 Standings and leakage checks . . . 12
7.4 Exploratory evidence . . . 12
7.5 Empirical selection: adding one group at a time . . . 14
7.6 Statistical check of the selection . . . 14

8 Preprocessing **15**

9 Models **16**

10 Results **17**

11 Experiments **18**
11.1 Feature-group ablation . . . 18
11.2 Preprocessing experiments . . . 19
11.3 Why not the window with the best validation score? . . . 20

12 Explainability (XAI) **20**
12.1 Global: SHAP summary plots . . . 21
12.2 Local: SHAP force plots and LIME . . . 21

13 Inference functions **24**

14 Limitations **24**

15 Conclusion **25**

A Reproducibility and methods **25**

1 Task and dataset

Task. The unit of analysis is one driver entering one race, identified by `(raceId, driverId)`. The target is `podium = 1` if `positionOrder` ≤ 3, otherwise 0. The prediction is made **after qualifying and before the start**. Anything known only during or after the race (finishing position, points, laps, race time, fastest lap, retirement status, standings after the race) would be *leakage* and is never an input.

Dataset. The Kaggle/Ergast *Formula 1 World Championship (1950–2024)* dataset [6]: 14 relational CSV tables. `results` (26,759 rows) is the central table; its ID columns point to `races`, `drivers`, `constructors` (teams) and `status`, and `races` points to `circuits`. The raw files are never modified: every cleaning step works on copies in memory.

Tools. pandas, numpy, matplotlib, seaborn and missingno for data handling and plots; scikit-learn [5] for the statistical models; TensorFlow/Keras for the neural network; shap and lime for the explanations. The notebook runs top to bottom with one *Run All* on Kaggle (CPU only).

2 Exploratory data analysis before cleaning

2.1 First look and sentinels

`head`, `info` and `describe` on `results` show the main problem at once: `position`, `milliseconds`, `fastestLap`, `rank` and `fastestLapSpeed` hold numbers but were read as text, and `isnull()` reports no missing values. The dataset writes "no value" as the string `\N` (a *sentinel*), which hides missing values from pandas. Over all tables there are 160,076 sentinel cells in 30 columns, and `isnull()` finds only 68 of them *(notebook §4)*.

Table 1: Columns with the most `\N` sentinels.

Table	Column	Sentinels	Share of rows
results	time, milliseconds	19,079	71.3%
results	fastestLap, fastestLapTime, fastestLapSpeed	18,507	69.2%
results	rank	18,249	68.2%
constructor_results	status	12,608	99.9%
results	position	10,953	40.9%
qualifying	q3 / q2	6,819 / 4,625	65.0% / 44.1%
races	sprint_date	1,107	98.4%

2.2 Where and when values are missing

Figure 1 shows the missing values of `results` and `qualifying`, and Figure 2 shows how coverage changes over the 75 seasons. The first season with any value is 1994 for Q1 times, 2005 for Q2, 2006 for Q3 and 2004 for fastest-lap times.

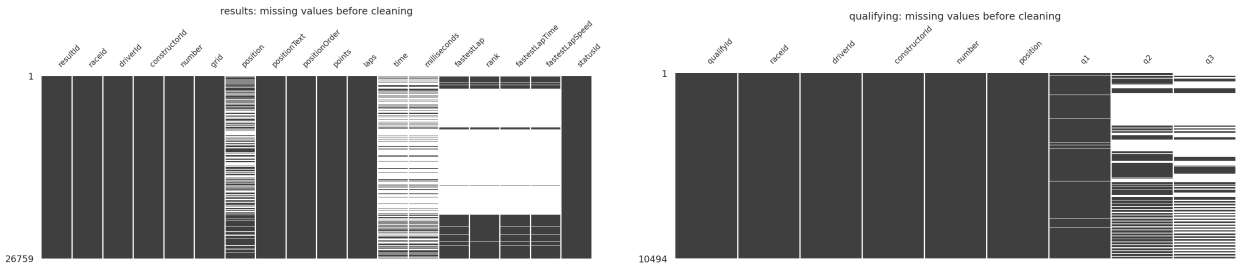


Figure 1: Missing values before cleaning (white = missing). In `results` they sit in the columns that describe how a race ended.

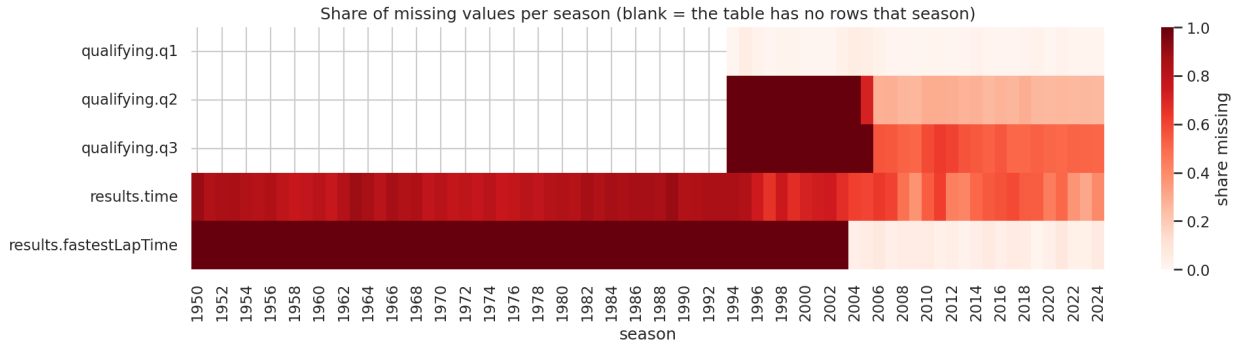


Figure 2: Share of missing values per season (blank = no rows that season). Even after 2006, Q2/Q3 are partly empty because only the faster drivers reach them (28%/55% missing in 2014, 25%/51% in 2024).

2.3 Spelling, grid slots and field sizes

Every name-like text column was checked with the same rules: spaces at the start or end, HTML codes, and values that become identical once case, accents and punctuation are ignored. Nationalities and circuit countries (compared in Q2) got extra checks for compound values, abbreviations and shared first letters. Found: `'Argentinian '` (with a trailing space) next to `Argentine`; `United States` next to `USA`; two compound nationalities (`American-Italian`, `Argentine-Italian`); `East German`; and one HTML code (`Lotus-Pratt & Whitney`). Four first names differ only by an accent (`Adrian`/`Adrián`), but they belong to different drivers and no analysis uses first names, so they are left alone *(notebook §4.1)*.

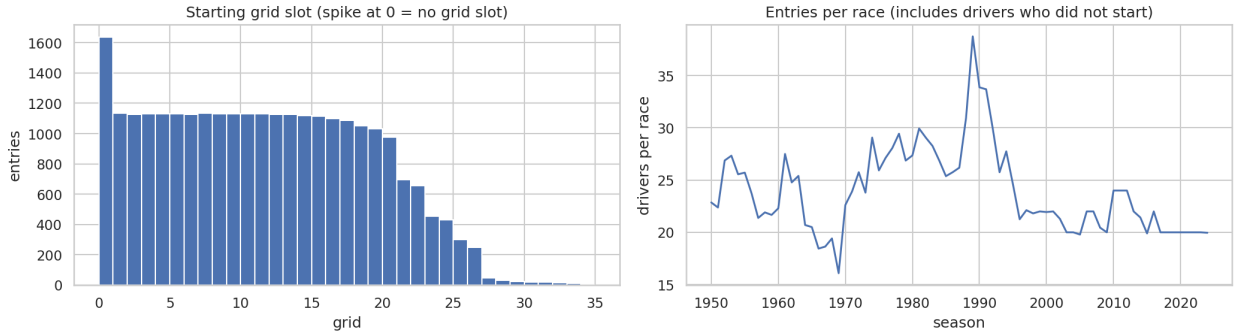


Figure 3: Left: starting grid slot, with a spike at 0. Right: entries per race, including drivers who never started.

`grid = 0` occurs 1,638 times and means two things: 72 pit-lane starters (who drove laps) and entries that never started (Figure 3). Until the 1990s many more drivers entered a race than could start.

2.4 Keys and duplicates

- **Primary keys** are found by rule, not typed in: the first column if it is an ID and unique; for `lap_times` and `pit_stops` the lap or stop counter is added to `(raceId, driverId)`. All 14 keys are unique and never empty.
- **Foreign keys** are every column named like another table's key. All 23 links have 0 orphans. The notebook stops if either check fails *(notebook §5)*.
- **Duplicates.** 85 `(raceId, driverId)` pairs appear more than once (176 rows), all between 1950 and 1978. In all 85 pairs the rows have different car numbers: these are *shared drives*, where a driver drove more than one car in the same race *(notebook §5.1)*.

Duplicate rule. We keep the row with the **lowest real grid slot**, taken as the car the driver started in; ties (1 pair) and pairs without a slot (3 pairs) are decided by the better finish. The rule matters for the label: keeping the best finish would give 22 podiums instead of 5, because a driver who took over a teammate's car would be credited with that car's result. Our rule keeps the car that the pre-race input (`grid`) describes. All 85 pairs are in training seasons.

3 Cleaning

All cleaning works on copies. Missing values are *not* filled here; filling happens later with training statistics only (Section 8).

Table 2: Cleaning steps and their effect *(notebook §6)*.

Step	What it does and its effect
1. Sentinels	`\N` → NaN (160,076 cells); spaces trimmed; HTML codes decoded; text columns whose values are all numbers become numbers (6 in `results`, 1 in `drivers`, 3 in `sprint_results`).
2. Semantic typing	Dates → `datetime`; lap times and durations (`"1:27.452"`) → seconds. The parsed lap and pit-stop times match the dataset's own millisecond columns exactly (asserted).
3. One spelling per value	The more common spelling is kept (`United States` → `USA`, `Argentinian` → `Argentine`); compound nationalities keep the first; `East German` → `German`. Every driver gets a `home_country` in the circuits' spelling (asserted complete); 14 home countries never hosted a race.
4. Unused columns	Web links and practice/qualifying/sprint session dates dropped.
5. One row per driver-race	The duplicate rule above: 26,759 → 26,668 rows; uniqueness asserted.
6. Flags	1,354 entries whose status is "Did not qualify" or "Did not prequalify" are removed from modeling and Q3 (12 of them still carry a grid number, a data error). 221 entries qualified but did not start: kept, as honest "no" rows (0 podiums). 72 pit-lane starters kept. Entries without a slot get the slot behind the last car (`grid_eff`). 381 Indianapolis 500 rows (1950–1960) flagged and excluded.
7. Status classes	137 statuses grouped by reading them: finished (15,115 rows), mechanical (6,775), accident (2,771), regulatory (1,942), driver (65). Nine statuses that do not say what failed (`Retired`, `Puncture`, `Out of fuel`, ...) count as mechanical and are flagged as unclear. Used in Q3.

Evidence for the rules *(notebook §6.1)*. Pit-lane starters finish like the last three grid slots (median finishing position 15 for both; 0 podiums in 72 pit-lane starts), so "one slot behind the last car" fits. Of 381 Indy 500 driver-seasons, only 2 (1%) also raced another championship race that season, so it was a separate race with its own drivers.

Effect of the cleaning. The former sentinel columns are numbers, and missing values are visible to pandas. Removing the entries that did not qualify brings the number of drivers per race down to the real fields (Figure 4).



Figure 4: After cleaning. Top: missing cells per table before and after (log scale). Bottom: grid slots, drivers per race before and after removing non-qualifiers, and Q1 lap times after parsing to seconds.

4 Target, pre-race cutoff and metrics

Every driver who qualified is counted, including the few who did not start: 13.3% of entries end on the podium (3,378 of 25,314), so the classes are imbalanced. The rate follows 3/starters per race (Figure 5), so it is about 10% in the large fields of the 1950s–80s and about 15% today. This era shift is one reason the data is split by season.

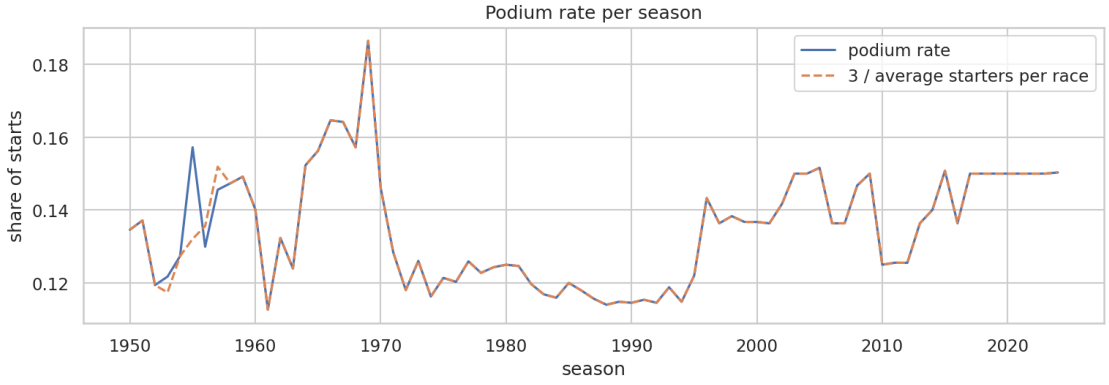


Figure 5: Podium rate per season and 3/average starters per race. The lines part only in a few 1950s races with shared drives.

Table 3: The pre-race cutoff.

Allowed (known before the start)	Not allowed (only known after the race)
`grid` (qualifying result after penalties)	`position`, `positionText`, `positionOrder`, `points`
driver, team and circuit identity	`laps`, `time`, `milliseconds`, `statusId`
championship standings **after the previous round**	`fastestLap`, `rank`, `fastestLapTime`, `fastestLapSpeed`
results of **earlier** races	this race's `lap_times`, `pit_stops`, and standings after this race

Sprints. A sprint result is known before Sunday's race, so it would be allowed. But only 18 of 1,125 races have a sprint, and sprints only started in 2021 (general knowledge), so none falls in the training seasons and the feature would be empty for every training row. Sprint results are not used.

Metric choice. A model that always says "no podium" is about 87% accurate and useless, so accuracy is only reported for reference. The **primary metric is PR-AUC** (average precision): how well the real podium finishers are ranked above everyone else, focused on the rare class. The **secondary metrics** are ROC-AUC (overall separation of the classes) and F1 at a threshold chosen on validation (the quality of the actual yes/no decision). Precision, recall and accuracy are reported next to F1.

5 Data-engineering questions

Each answer joins several tables, and after every join the notebook asserts that the number of rows did not change. The answers describe the data and are not used to choose anything in the modeling, so they may look at all seasons. Rates come with 95% Wilson intervals [8, 1]: the simple "rate $\pm$ 1.96 standard errors" breaks for small counts (0 of 12 gives $0\% \pm 0$; 1 of 12 gives a lower end of -7.3%), while Wilson's interval stays inside 0–100% and widens when there are few cases. If two intervals do not overlap, the difference is very unlikely to be chance.

5.1 Q1: Which circuits convert a front-row start into a podium most often, and did that change after 2014?

Method. `results` $\rightarrow$ `races` $\rightarrow$ `circuits`. **Grain:** one front-row start (grid slot 1 or 2), 2,259 rows, unique per `(raceId, driverId)`. **Qualifying vs. grid:** they can differ because of penalties; we use `grid`, the slot the car actually started from, which exists for every race since 1950 (qualifying positions start in 1994). **Missingness:** `grid` and `positionOrder` are never missing. **Cutoff:** the starting slot is known before the start; the podium is the outcome being counted. A circuit is ranked only with at least 10 front-row starts (about 5 races) in *each* era: a circuit used once has 2 front-row starts, so its rate can only be 0%, 50% or 100%. Of 77 circuits, 45 were raced only before 2014 and 8 only from 2014; of the 24 raced in both eras, 16 pass the minimum.

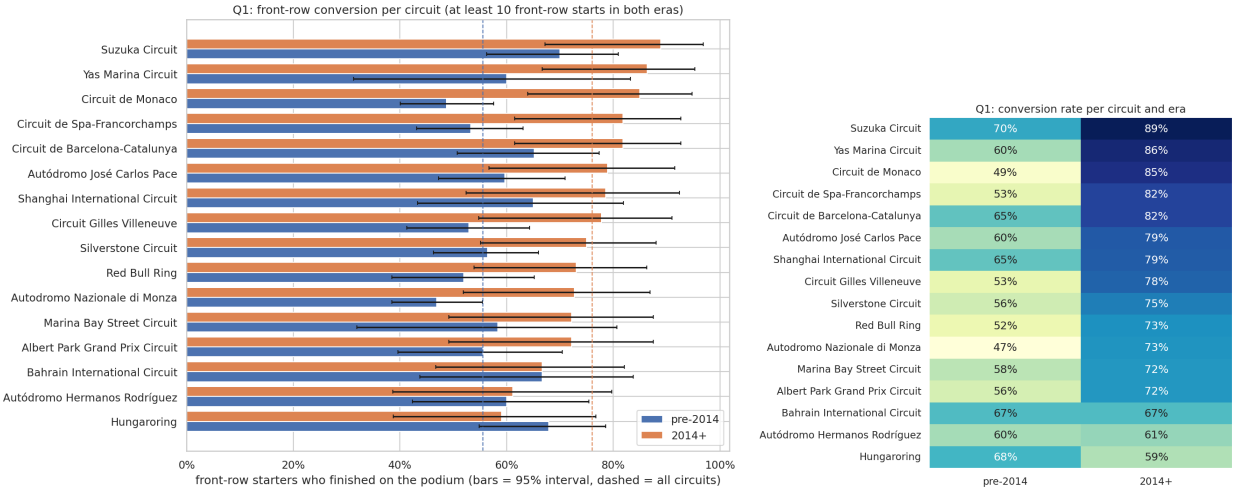


Figure 6: Q1: front-row conversion per circuit, before 2014 and from 2014 (bars = 95% intervals, dashed = all circuits), and the same rates as a heatmap.

Answer. A front-row start converts much more often in the hybrid era: **55.5% before 2014** (1,002 of 1,805; 53.2–57.8%) **vs. 76.0% from 2014** (345 of 454; 71.9–79.7%); the intervals do not overlap. The best circuits since 2014 are Suzuka (16 of 18, 89%), Yas Marina (86%) and Monaco (85%). The ranking is **not stable**: the Spearman rank correlation between the eras is −0.03 ($p = 0.91$, 16 circuits). Monaco went from 15th of 16 (49%) to 3rd, and the Hungaroring from 2nd (68%) to last (59%). With other minimums (6, 14 or 20 starts) the correlation stays between −0.18 and 0.06, and Suzuka stays on top for 6 and 14.

5.2 Q2: Do drivers podium more often at home, for the same starting position?

Method. `results` → `drivers` → `races` → `circuits`. A home race is a race in the driver's `home_country` (cleaning step 3). **Grain:** one qualified entry, without the Indy 500: 24,933 starts, 1,974 at home (7.9%). **Missingness:** every driver has a home country and every circuit a country. **Cutoff:** home status and starting position are known before the start. Home drivers might simply qualify better, so the rates are compared *within* starting groups (pole, 2–3, 4–6, 7–10, 11+), and the home-minus-away gaps are averaged, weighted by home starts. A logistic regression of podium on home, starting group and era (dummy columns, one category left out as reference, effectively no regularisation) gives the home effect as an **odds ratio**: 1 = no effect, below 1 = home does worse.

Table 4: Q2: podium rate by starting group, home vs. away.

Start	Away rate	Home rate	Home – away	Home starts	Away starts
Pole	64.2%	58.3%	−5.9 pp	84	1,040
2–3	50.9%	53.7%	+2.9 pp	147	2,078
4–6	26.0%	26.4%	+0.4 pp	227	3,114
7–10	9.2%	7.1%	−2.1 pp	336	4,117
11+	1.6%	1.0%	−0.6 pp	1,180	12,610

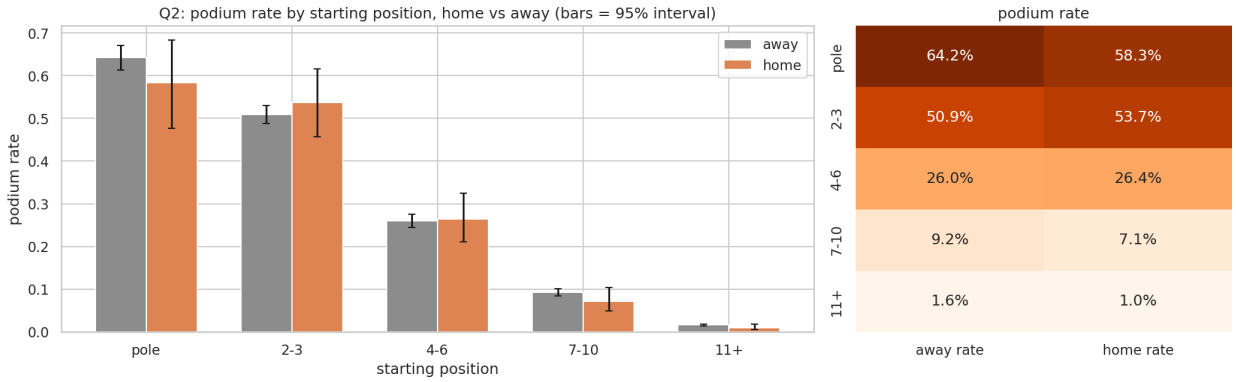


Figure 7: Q2: podium rate by starting position, home vs. away (bars = 95% intervals).

Answer. **No home advantage.** From the same starting group, home drivers are 0.70 percentage points *less* likely to finish on the podium, and the odds ratio is **0.912**. In every starting group the home and away intervals overlap, so the small negative gap cannot be told apart from no effect. The odds ratio stays between 0.894 and 0.912 with other era splits (none, 2014 only, decades).

5.3 Q3: Which constructors have the highest mechanical-retirement rate in 2014–2021 vs. 2022–2024?

Method. `results` → `status` (class from cleaning step 7) → `constructors`. **Grain:** one race start in 2014–2024 (4,619 rows); entries that never started are left out, because the brief defines a mechanical retirement as a race start that ended early. **Definition:** a part of the car failed; crashes, driver causes and rule decisions are excluded. **Missingness:** every entry has a status and so a class. **Cutoff:** Q3 describes how races ended, so it uses post-race data on purpose; none of it is a model input. A team is *clearly less reliable* than its period if even the low end of its interval is above the period's average. The periods differ in length, so rates are compared, never counts. Teams are compared under the name in the data; renames (Toro Rosso → AlphaTauri → RB, Sauber → Alfa Romeo → Sauber, ...) are general knowledge from the teams' Wikipedia pages, and the seasons per name in the data fit them.

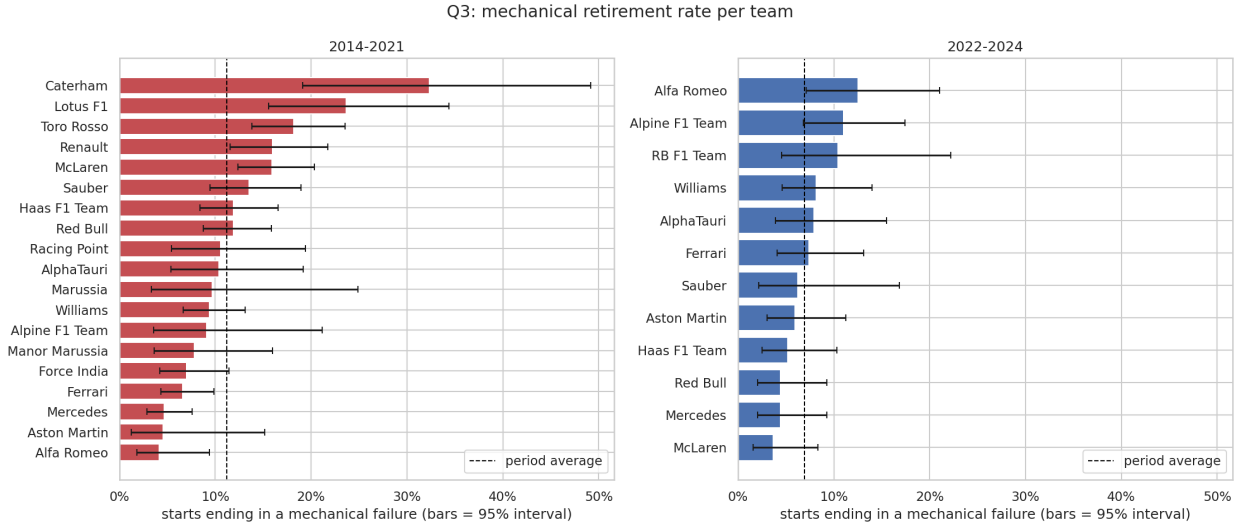


Figure 8: Q3: mechanical retirement rate per team and period (bars = 95% intervals, dashed = period average).

Table 5: Q3: highest mechanical-retirement rates per period.

Period	Team	Starts	Mechanical	Rate
2014–2021	Caterham	34	11	32.4%
2014–2021	Lotus F1	76	18	23.7%
2014–2021	Toro Rosso	242	44	18.2%
2022–2024	Alfa Romeo	88	11	12.5%
2022–2024	Alpine F1 Team	136	15	11.0%
2022–2024	RB F1 Team	48	5	10.4%

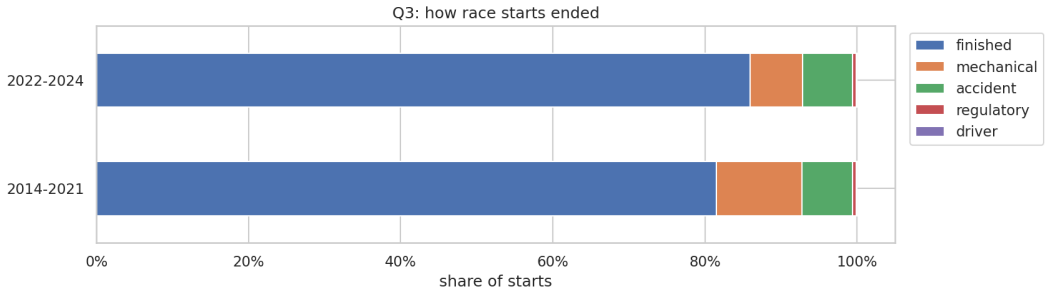


Figure 9: Q3: how race starts ended in each period.

Answer. Cars became more reliable: **11.2%** of starts in 2014–2021 (366 of 3,263; 10.2–12.3%) vs. **6.9%** in 2022–2024 (94 of 1,356; 5.7–8.4%) ended with a mechanical failure; the intervals do not overlap. The highest rates were Caterham, Lotus F1 and Toro Rosso in 2014–2021, and Alfa Romeo, Alpine and RB in 2022–2024 (Table 5). Caterham's 32% rests on 34 starts (interval 19–49%). Clearly less reliable than their period: Caterham, Lotus F1, McLaren, Renault and Toro Rosso in 2014–2021, and only Alfa Romeo in 2022–2024. Without the unclear statuses the rates fall to 9.7% and 6.0% and the top 3 stays the same, except that AlphaTauri takes RB's third place in 2022–2024.

6 Modeling table and data split

One row per driver who *qualified* for a race in 1950–2024 (including the few who did not start), without the Indy 500: **24,933 rows**, `(raceId, driverId)` unique (asserted). Entries without a grid slot start one slot behind the last car.

Table 6: Season-based split *(notebook §9)*.

Part	Seasons	Rows	Podiums	Rate	Used for
Train	1950–2017	21,954	2,895	13.2%	fitting models, imputer, scaler
Validation	2018–2021	1,620	243	15.0%	features, settings, thresholds
Test	2022–2024	1,359	204	15.0%	final scores, used once

Why split by season. The task is to predict future races. A random split would put drivers of the same race on both sides and train on seasons later than the ones scored, which makes scores look better than they are. A season split always scores the model on years it has never seen, and the test block is the newest set of car rules (2022–2024).

7 Feature engineering and selection

7.1 Candidate features and why they are valid at the cutoff

Table 7: Candidate features (11 in 5 groups). All use only information from before the race.

Group	Features	Why it is known before the start
grid	`grid`	Set by qualifying and penalties before the start (no slot = behind the last car).
field size	`field_size`, `grid_rel`	Cars in the race and grid ÷ cars; the field is fixed once the grid is.
driver standings	`drv_pos`, `drv_pts_share`, `drv_wins`	Position, points ÷ leader's points, and wins from the standings **after the previous round** of the same season.
team standings	`con_pos`, `con_pts_share`, `con_wins`	The same for the team.
recent form	`drv_form`, `con_form`	Podium share in the driver's (team's) last `FORM_WINDOW` races *before* this one; for a team, the share of its cars.

There are no qualifying-time features: qualifying times exist only from 1994, and the grid already holds the qualifying result after penalties. Points are a share of the leader's points, because raw points are not comparable across points systems (Figure 10). Driver and team identity enter only through these features; one-hot encoding 861 driver IDs would add hundreds of mostly empty columns, and many test drivers never appear in training.

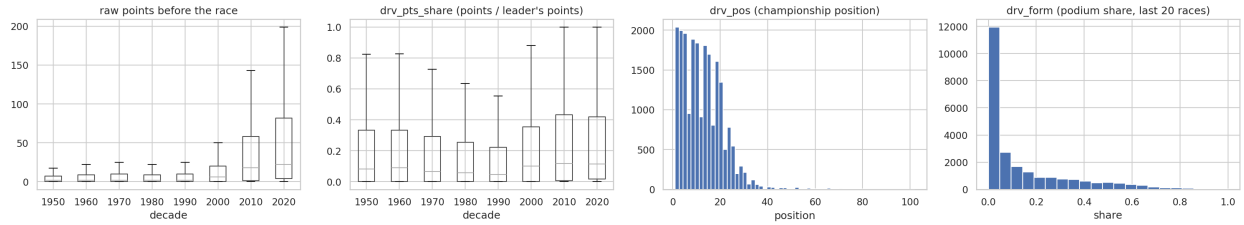


Figure 10: Engineered features on all rows: raw points and the points share per decade (the share always runs from 0 to 1), championship positions, and driver form.

7.2 How many races should recent form look back?

For windows of 1 to 150 races, the two form features are built and a logistic regression on `grid` + both features is scored on validation *(notebook §9.1)*. One validation score is not exact, so the rule is: **the shortest window that validation cannot tell apart from the best one**, because a shorter window reacts faster when the order of the teams changes. Whether two windows can be told apart is checked with a **race bootstrap** [3]: the validation races are drawn at random with repeats (whole races, because the drivers of one race share one result), both windows are rescored on the drawn rows, and this is repeated 2,000 times; the middle 95% of the differences is the 95% range of the gap. If it reaches 0, validation cannot tell the two windows apart.

Table 8: Form window: Spearman correlation of each form feature with the podium (train), validation PR-AUC of a logistic regression on grid + both form features, and its gap to the best window with the 95% bootstrap range *(notebook §9.1)*.

Window	Driver form ρ	Team form ρ	Val PR-AUC	Gap to best (95% range)	As good as best
1	0.330	0.360	0.681	−0.049 (−0.084 to −0.014)	no
3	0.414	0.406	0.692	−0.038 (−0.070 to −0.005)	no
5	0.434	0.412	0.691	−0.039 (−0.067 to −0.009)	no
10	0.434	0.411	0.699	−0.031 (−0.056 to −0.002)	no
20	0.422	0.403	0.710	−0.020 (−0.035 to +0.001)	**yes (chosen)**
30	0.410	0.393	0.715	−0.015 (−0.032 to +0.006)	yes
50	0.394	0.379	0.729	−0.001 (−0.014 to +0.011)	yes
75	0.381	0.367	**0.730**	best	yes
100	0.374	0.362	0.724	−0.007 (−0.017 to +0.006)	yes
150	0.369	0.353	0.726	−0.004 (−0.019 to +0.011)	yes

The best validation window is 75 races, but every window from 20 races up cannot be told apart from it, while 1–10 races are clearly worse. So `FORM_WINDOW` = **20 races**. Section 11.3 shows what 75 races would have done on test.

7.3 Standings and leakage checks

- **The standings of a race already include that race.** A driver's points at a race equal the previous round's points plus the points scored in that race (sprints included) in 99.5% of rows for 1950–1990 and 100% for 1991–2024. That is why the features take the standings of the *previous* round *(notebook §9.3)*.
- **Why standings are missing.** Round 1 has no previous round (1,493 train, 80 validation, 60 test rows); a driver's first race has no standings (452 / 2 / 5); a driver can be absent from the previous round's standings (905 / 2 / 3); and there was no constructors' championship before 1958 (1,033 rows). Share missing: driver standings 13.0% of train, 5.2% of validation, 5.0% of test.
- **Recent form uses only earlier races.** For 500 random rows the form is recomputed by hand from the driver's (team's) earlier races only; it equals the feature in every row (asserted). One driver's races are also printed as a worked example *(notebook §9.4)*.
- **Standings come from an earlier race of the same season** for all 23,300 rows that have them, and the 1,633 round-1 rows have none (asserted).

7.4 Exploratory evidence

For each candidate, the training rows are cut into about five groups by value (equal values never split; a value with more than a fifth of the rows, such as 0 wins, is its own group) and the podium rate of each group is plotted (Figure 11). Spearman is used instead of Pearson because the relations are not straight lines: for `grid`, the podium rate drops steeply over the first slots and then flattens.

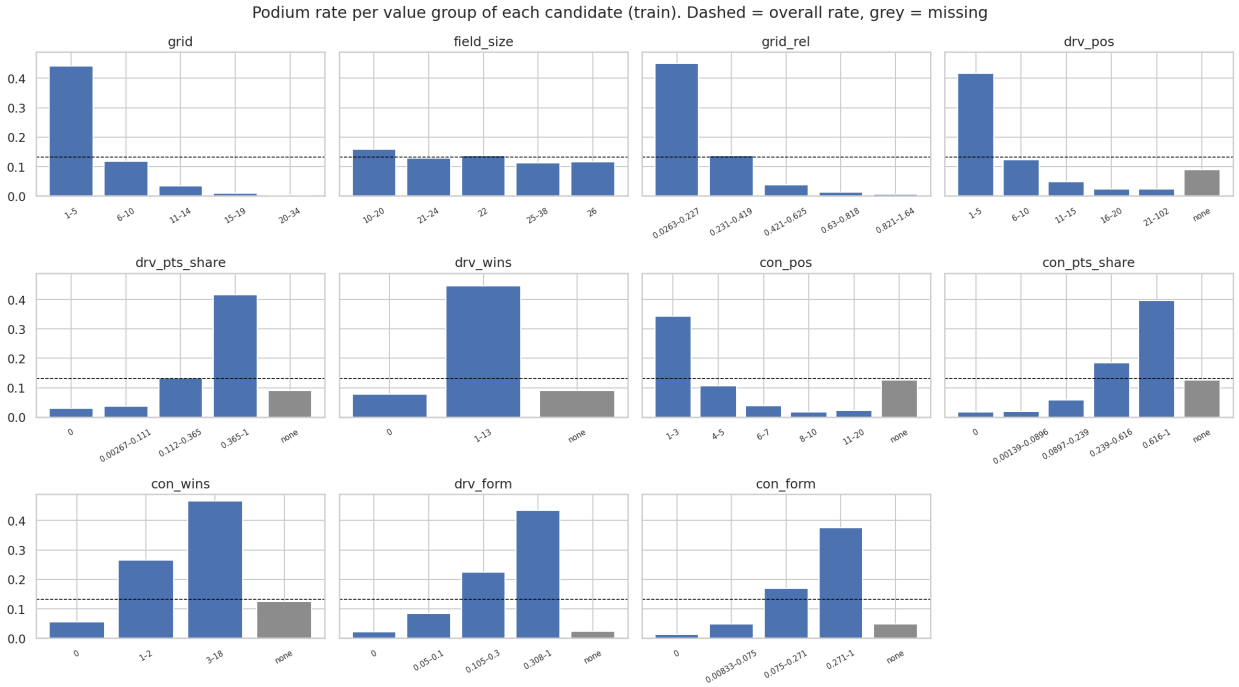


Figure 11: Podium rate per value group of each candidate (train). Dashed = overall rate; grey = missing value.

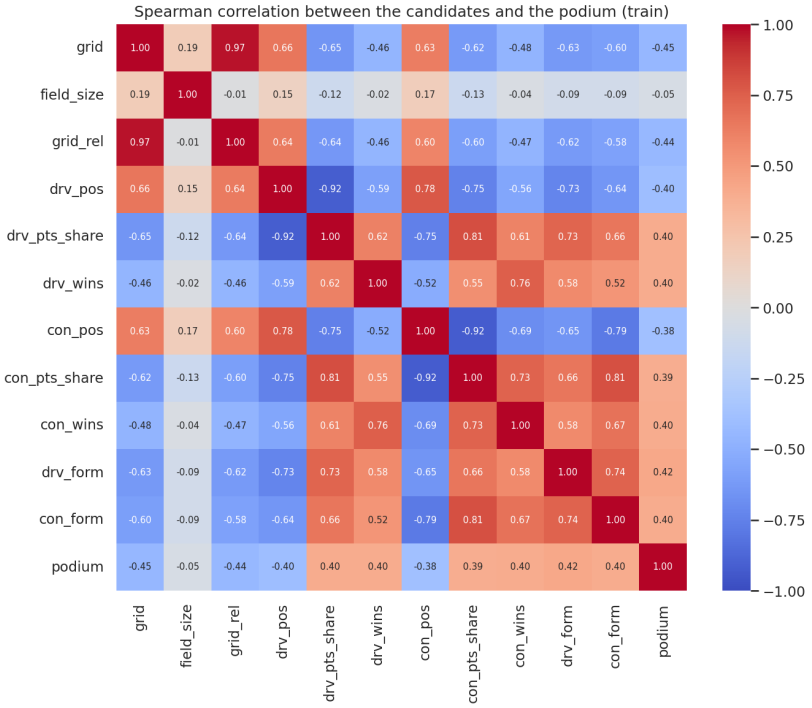


Figure 12: Spearman correlation between the candidates and the podium (train; last row and column = podium).

Spearman correlation with the podium (train): `grid` −0.449, `grid_rel` −0.444, `drv_form` 0.422, `drv_wins` 0.404, `con_form` 0.403, `drv_pts_share` 0.400, `drv_pos` −0.400, `con_wins` 0.400, `con_pts_share` 0.393, `con_pos` −0.383, `field_size` −0.052. All but field size are strong on their own, but many repeat each other (grid and relative grid, position and points share, driver and team standings, form and standings), so the plots alone cannot settle the list.

7.5 Empirical selection: adding one group at a time

Starting from `grid` only (the baseline), the groups are added one at a time in the order of Table 7. Logistic regression and a random forest are retrained with fixed settings, so only the features change. **A group is kept if validation PR-AUC rises by at least 0.005 for both models**; a dropped group is never carried along. An extra group makes the models harder to explain and gives them more ways to fit noise, so it has to bring a gain worth having; 0.005 is that bar, and the bootstrap below checks it.

Table 9: Feature-selection steps (validation PR-AUC) *(notebook §11.2)*.

Step	Groups in this step	LR	RF	Decision
grid	grid	0.668	0.668	start
+ field size	grid + field size	0.668	0.668	dropped
+ driver standings	grid + driver standings	0.714	0.723	**kept**
+ team standings	grid + driver + team standings	0.706	0.704	dropped
+ recent form	grid + driver standings + form	0.716	0.730	dropped
all 11 candidates		0.719	0.720	for comparison

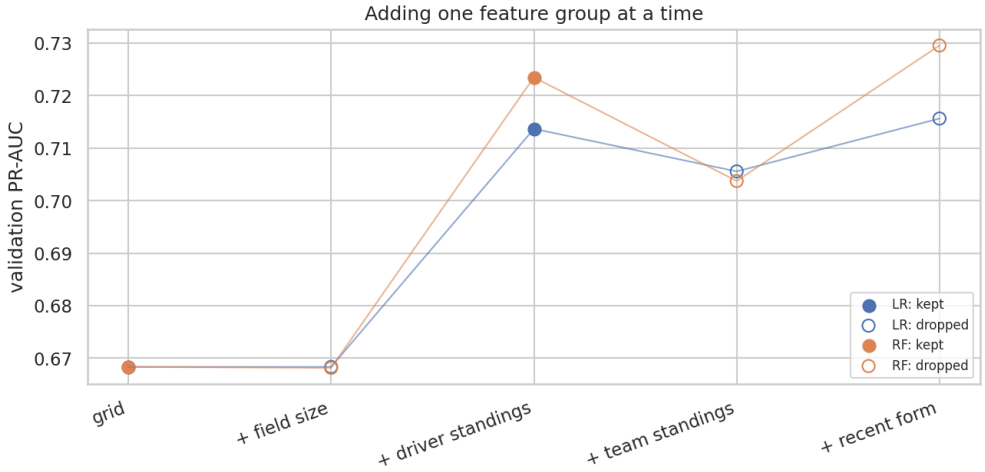


Figure 13: Validation PR-AUC as groups are added (filled = kept, open = dropped).

Selected features (4): `grid`, `drv_pos`, `drv_pts_share`, `drv_wins`. Driver standings are the only group that clears the bar: how the driver is doing *this season* is the strongest addition to where they start. Field size adds nothing; team standings make both models worse (-0.008, -0.020); recent form helps the forest a little ($+0.006$) but not logistic regression ($+0.002$), and the rule asks for both. All 11 candidates together change the score by $+0.005$ (LR) and -0.003 (RF) compared with the selected four, so seven more columns are not clearly better. The result can depend on the order of the groups, and a dropped group is not tried again; the all-candidates row and the ablation (Section 11.1) are the checks on that.

7.6 Statistical check of the selection

Redundancy. The variance inflation factor (VIF) of a column is $1/(1 - R^2)$, where R^2 says how well the other columns predict it; above about 5 a column is mostly a copy. With all 11 candidates, `grid` (62.7) and `grid_rel` (59.5) are copies of each other, and the points shares of the driver (6.4) and the team (7.2) mostly repeat the other standings columns. The selected four have VIFs of 1.6–3.5. All Spearman p-values are below 0.001, but with 22,000 rows even $\rho = -0.05$ is "significant", so the size of the correlation and the redundancy decide, not the p-value.

Each step against chance. The same 2,000 bootstrap redraws of the validation races as in Section 7.2 are used: both versions of each step are rescored on the same drawn rows (no retraining).

Table 10: Change in validation PR-AUC for each step, with its 95% bootstrap range *(notebook §11.2b)*.

Step	Model	Change	95% range	0.005 rule
+ field size	LR / RF	+0.000 / −0.000	0.000 to 0.000 / −0.001 to +0.000	dropped
+ driver standings	LR	+0.045	+0.009 to +0.085	**kept**
	RF	+0.055	+0.017 to +0.093	**kept**
+ team standings	LR	−0.008	−0.020 to +0.004	dropped
	RF	−0.020	−0.040 to +0.002	dropped
+ recent form	LR	+0.002	−0.023 to +0.029	dropped
	RF	+0.006	−0.016 to +0.030	dropped

The bootstrap agrees with the 0.005 rule at every step: only the kept step has a range above 0 for both models, and every dropped step has a range that includes 0.

8 Preprocessing

1. **Imputation** with the **median** of the training rows (`SimpleImputer`). The columns with gaps are skewed (skew 1.0–4.5 for all but `con_pos`; 56–84% of rows lie at or below the mean), so the mean is pulled up by a few strong drivers and is a value hardly anyone has, while the median is what a typical driver has (Figure 14). Fill values: `grid` 12, `drv_pos` 12, `drv_pts_share` 0.076, `drv_wins` 0.
2. **Scaling** with `StandardScaler` (mean 0, standard deviation 1), so large numbers (grid 1–34, positions up to 102) do not dominate small ones (shares 0–1). It matters for logistic regression and the network; the forest does not care, but every model gets the same inputs.

Both steps are fit on the **training seasons only** and applied unchanged to validation and test. No row is dropped for a missing value: 68 of the 1,359 test rows (5.0%) have missing standings and all are scored. Scaling only shifts and stretches the values (Figure 15). Validation and test are not refit, so their means differ from 0 (for example `grid` −0.22 and −0.23), which shows how the newer seasons differ from the training seasons.

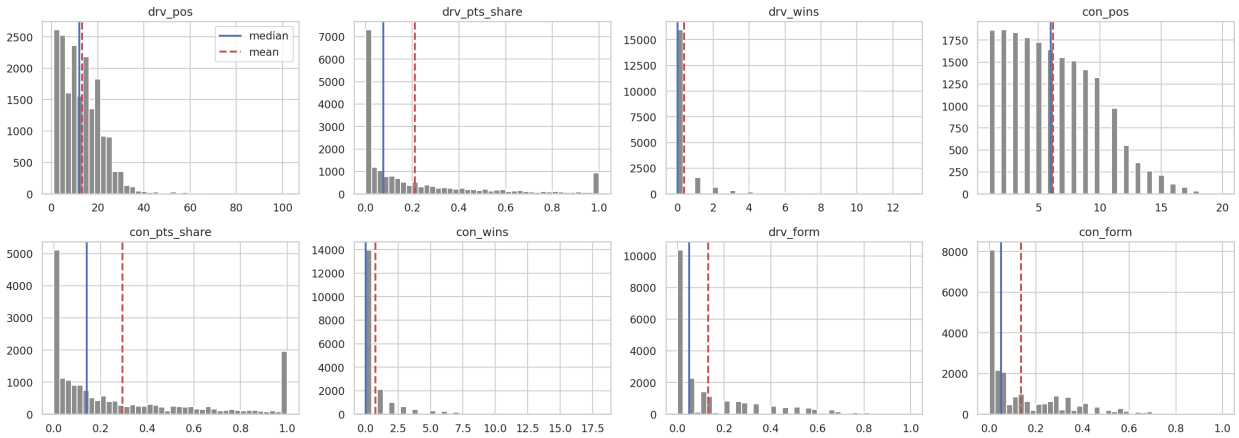


Figure 14: The training columns with gaps; solid = median (the fill value), dashed = mean.

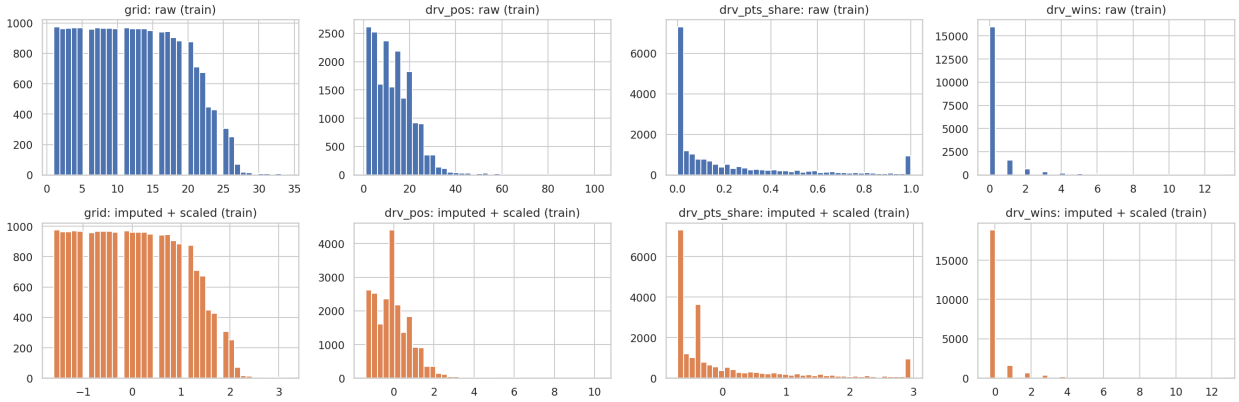


Figure 15: The four chosen features on train, raw (top) and after imputing and scaling (bottom).

9 Models

All models use the same rows and the same four inputs, so their scores are directly comparable. No single model is picked as "final"; all three are reported and explained alike.

- **Baseline: logistic regression on `grid` only**, the simplest reasonable model. The others must beat it to earn their complexity.
- **Logistic regression:** a weighted sum of the inputs passed through a sigmoid. Easy to read, but it can only learn effects that keep going in one direction.
- **Random forest** [2]: 300 decision trees, each on a random sample of rows and features, averaged. It learns curves and interactions, but is harder to read.
- **Shallow neural network** (Keras): one or two hidden `Dense` ReLU layers, one sigmoid output, binary cross-entropy, Adam (learning rate 0.001), batches of 256, at most 200 epochs, early stopping on validation loss (patience 15, best weights kept). Flexible, but a black box that depends on scaling and random initialisation.

Tuning (validation only). Every combination of class weight (none / `balanced`) and settings was trained, and the best validation PR-AUC kept (Table 11). The threshold of each model is the one with the best F1 on validation: baseline 0.427, logistic regression 0.399, random forest 0.432, network 0.348.

Table 11: Tuning (validation PR-AUC). Chosen settings in bold.

Model	Settings	No class weight	Balanced
LR	$C = 0.01$ / 0.1 / 1 / 10	0.711 / 0.714 / 0.714 / **0.714**	0.711 / 0.711 / 0.711 / 0.711
RF	depth 4, min leaf 20 / 50	0.722 / 0.715	0.715 / 0.708
RF	depth 6, min leaf 20 / 50	**0.723** / 0.720	0.722 / 0.717
RF	depth 8, min leaf 20 / 50	0.722 / 0.718	0.723 / 0.719

Table 12: Neural-network settings (validation).

Setting	Epochs run	Best epoch	Stopped early	Best val loss	Val PR-AUC
(16)	200	187	no	0.2265	**0.726**
(32)	200	200	no	0.2252	0.726
(32, 16)	130	115	yes	0.2242	0.723
(16) + dropout 0.2	102	87	yes	0.2325	0.716
(32, 16) + dropout 0.2	60	45	yes	0.2276	0.721
(16) + balanced	200	200	no	0.4201	0.726

The best setting of each model uses no class weight. C barely matters for logistic regression. The three network sizes without dropout are within 0.003 of each other and the smallest (16 units) scores highest, so it is kept: with four inputs a bigger network gains nothing (Figure 16).

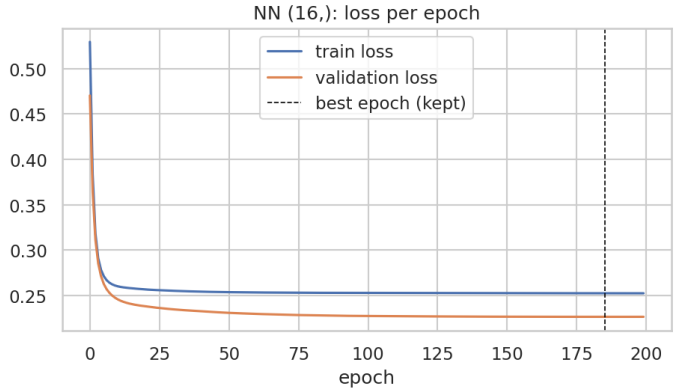


Figure 16: Network (16): training and validation loss per epoch; dashed = the epoch whose weights are kept.

10 Results

Table 13: All metrics on train, validation and the unseen test seasons. Thresholds were chosen on validation *(notebook §14)*.

Model	Set	PR-AUC	ROC-AUC	F1	Precision	Recall	Accuracy
Baseline (LR on grid)	train	0.487	0.883	0.534	0.533	0.535	0.877
	val	0.668	0.918	0.712	0.712	0.712	0.914
	test	0.647	0.914	0.649	0.650	0.647	0.895
Logistic regression	train	0.545	0.892	0.535	0.565	0.507	0.883
	val	0.714	0.928	0.709	0.702	0.716	0.912
	test	0.708	0.930	0.678	0.690	0.667	0.905
Random forest	train	0.567	0.901	0.543	0.586	0.505	0.888
	val	0.723	0.939	0.720	0.715	0.724	0.915
	test	0.720	0.940	0.668	0.709	0.632	0.906
Neural network	train	0.549	0.896	0.566	0.517	0.626	0.873
	val	0.726	0.936	0.716	0.655	0.790	0.906
	test	0.706	0.938	0.681	0.614	0.765	0.893

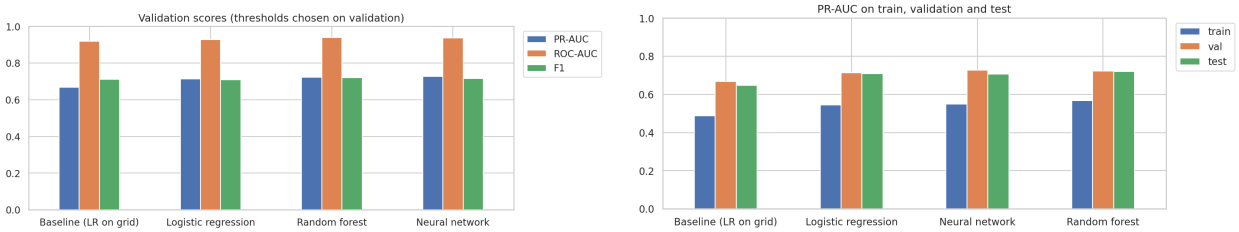


Figure 17: Left: validation PR-AUC, ROC-AUC and F1. Right: PR-AUC on train, validation and test.

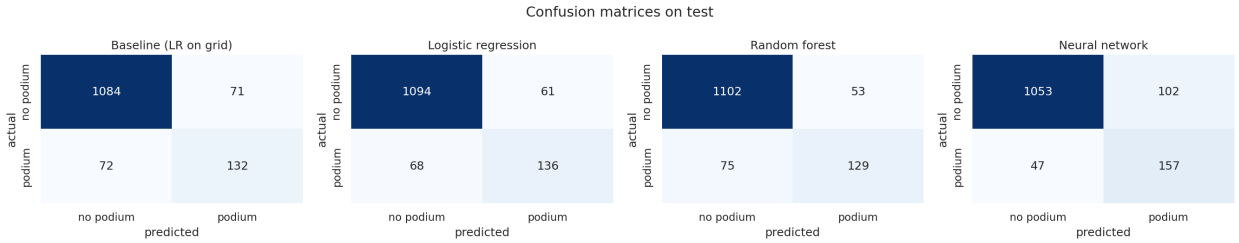


Figure 18: Confusion matrices on the test seasons (rows = actual, columns = predicted).

Reading the results.

- **All three models beat the baseline** on validation and test. On test, PR-AUC rises from 0.647 to 0.706–0.720 and ROC-AUC from 0.914 to 0.930–0.940: adding the driver's season standings to the grid earns its complexity.
- **The three models are about equally good.** The network is first on validation (0.726), the forest on test (0.720), and the gaps are about 0.01, which a few hundred podiums cannot separate. Test scores are within 0.02 of validation.
- The network has the highest test recall (0.77) and the lowest precision (0.61), because its threshold (0.348) is the lowest; F1 is 0.67–0.68 for all three.
- **Train is lower than validation** (PR-AUC 0.55–0.57 vs. 0.71–0.73), which is neither under- nor overfitting. Split by era *(notebook §13.1)*, train PR-AUC is 0.41–0.47 in 1950–1979 but 0.69–0.77 in 2014–2017, close to validation, while the podium rate is about the same in each era (13–14%). The old races are simply harder to predict.
- **The outputs are on the right scale:** every model's average prediction on validation (0.149–0.159) is close to the real rate (0.150), and none of the chosen settings uses `balanced` weights.

11 Experiments

Every experiment retrains all three models with their chosen settings, so only the tested thing changes.

11.1 Feature-group ablation

One group at a time is removed and the models are retrained. This is done from the selected features (what each kept group contributes) and from all 11 candidates (so every group gets a result). Test drops are only reported.

Table 14: Drop in PR-AUC when a group is removed (validation / test). Negative = better without it.

From	Removed group	Logistic regression	Random forest	Neural network
selected 4	grid	0.078 / 0.143	0.092 / 0.161	0.097 / 0.143
	driver standings	0.045 / 0.061	0.055 / 0.073	0.058 / 0.059
all 11	grid	−0.001 / −0.002	−0.003 / 0.010	0.007 / 0.008
	field size	0.001 / 0.001	0.005 / 0.013	0.027 / 0.024
	driver standings	0.009 / 0.007	0.006 / 0.003	−0.001 / −0.001
	team standings	0.004 / −0.001	−0.013 / −0.000	−0.002 / 0.005
	recent form	0.013 / 0.011	0.003 / 0.004	0.015 / 0.002

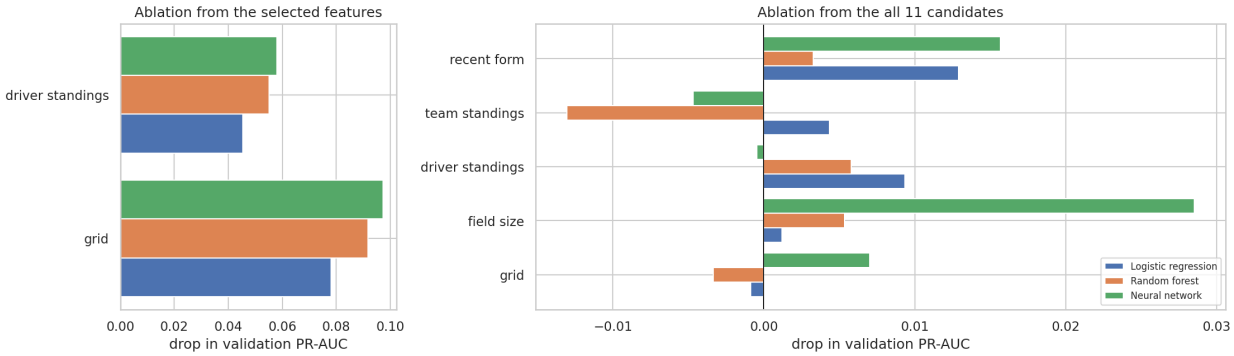


Figure 19: Drop in validation PR-AUC per removed group; left from the selected features, right from all 11 candidates.

Reading. From the selected features, removing the grid costs the most (0.08–0.10 on validation, 0.14–0.16 on test) and removing the driver standings costs 0.05–0.06 on validation and 0.06–0.07 on test, so both kept groups help after training too. From all 11 candidates every removal costs little (at most 0.027) and several make a model better: even removing the grid costs almost nothing, because `grid_rel` carries nearly the same information. The groups repeat each other, which is why only two were kept.

11.2 Preprocessing experiments

Table 15: Preprocessing experiments (validation PR-AUC) *(notebook §15.2)*.

Version	LR	RF	NN
Used: fit on train only, impute → scale, median	0.7137	0.7234	0.7263
1. Fit on all seasons (leaky)	0.7138	0.7229	0.7248
2. Scale → impute	0.7138	0.7234	0.7268
3a. Mean instead of median	0.7150	0.7227	0.7252
3b. Median + "was missing" flag	0.7170	0.7238	0.7281

- **Experiment 1 (before or after splitting):** fitting the imputer and scaler on all seasons is leakage, but it changes PR-AUC by at most 0.0015, because 88% of all rows are training rows, so the statistics hardly move. Fitting on train only is still the correct order: it is the only one possible for a real future race.
- **Experiment 2 (impute or scale first):** at most +0.0005. The filled value is the same point either way; imputing first is kept so the scaler sees exactly the data the model gets.
- **Experiment 3 (fill value):** the mean is no better (+0.0013, −0.0007, −0.0011). The missing flag is slightly better for all three models (+0.0032, +0.0004, +0.0018), but every

gain is below the 0.005 bar, so the plain median stays and the models keep four inputs.

11.3 Why not the window with the best validation score?

Section 7.2 uses the shortest window that validation cannot tell apart from the best one (20 races) instead of the best one itself (75 races). This section shows what the longer window does on the test seasons; test is only reported, and nothing is chosen from it. **Hypothesis:** a long window remembers several seasons. The validation seasons were one stable era led by Mercedes (45% of validation podiums), so a long memory predicted well there; with the new car rules of 2022 (general knowledge) the order at the front changed, and a long window kept ranking the old order first.

Table 16: The window model (LR on grid + form) on validation and test; test is only reported.

Window	1	3	5	10	**20**	30	50	75	100	150
Val PR-AUC	0.681	0.692	0.691	0.699	0.710	0.715	0.729	0.730	0.724	0.726
Test PR-AUC	0.663	0.702	0.695	0.710	0.710	0.697	0.680	0.658	0.633	0.603

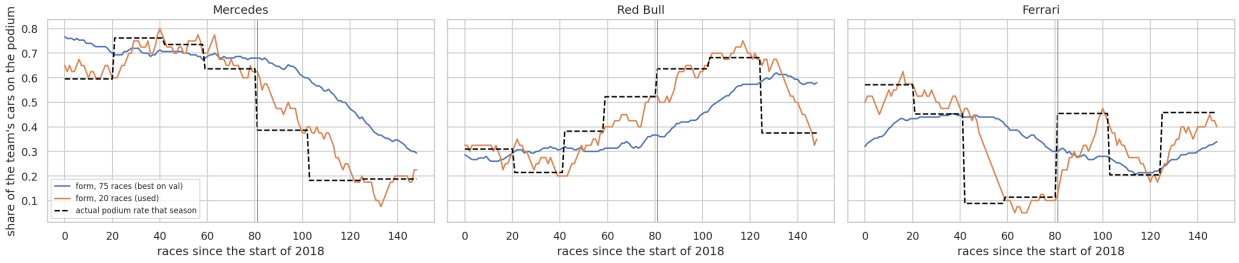


Figure 20: Team form of Ferrari, Mercedes and Red Bull from 2018, with 75 and 20 races, and the real podium rate of each season (dashed); grey line = first race of 2022.

The long windows are best on validation but worst on test (0.658 for 75 races, 0.603 for 150, against 0.710 for 10–20). In 2022 the 75-race form still put Mercedes far ahead of Red Bull (0.65 against 0.41) while Red Bull's cars actually reached the podium more often (0.64 against 0.39); the 20-race form had already switched the order (0.59 against 0.49). The short window lags too, by about a season (Red Bull 0.54 in 2024 against a real 0.38). This fits the hypothesis, but it rests on three teams and three seasons.

12 Explainability (XAI)

Explanations are not causes. An important feature is one the model *relies on to make predictions*. It is not evidence that the feature *causes* podium finishes. A front grid slot, for example, is mostly a sign of a fast car, and the fast car is what produces podiums. Inputs that overlap (the three standings columns) share the credit, and how they share it depends on the method.

Setup *(notebook §16).* The brief asks for permutation importance and SHAP globally, and LIME and/or local SHAP for one row. We use SHAP's **permutation explainer** [4] for all three models (the course staff confirmed this covers "permutation importance"): it switches inputs off in random orders, replacing them with values from 100 training rows, and records how far the predicted probability moves. The forest is also explained with `TreeExplainer`, which computes the values exactly from the trees. SHAP values are in probability units and add up to *prediction − average prediction* (asserted within 0.005). The rows are validation rows (2018–2021), and the inputs are shown in their original units. The baseline has one input, so there is nothing to split.

12.1 Global: SHAP summary plots

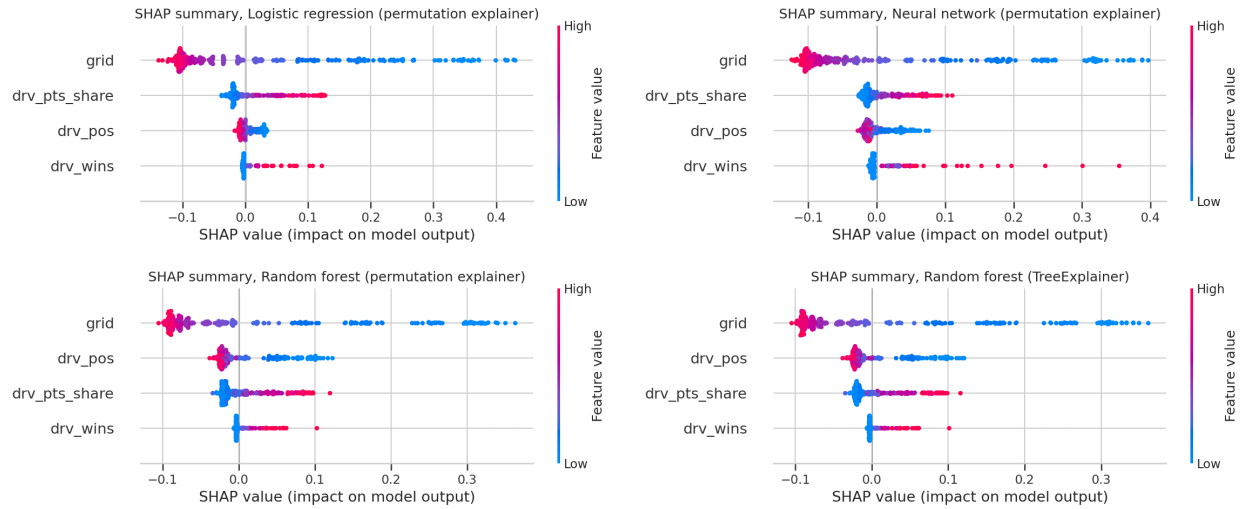


Figure 21: SHAP summary plots on 300 validation rows: logistic regression and network (top), forest with the permutation explainer and with `TreeExplainer` (bottom). One dot per driver-race; right = higher podium probability; colour = the input's value (red high, blue low).

Table 17: Average size of each input's SHAP value (probability units), 300 validation rows.

Input	Logistic regression	Random forest	Neural network	Forest (`TreeExplainer`)
`grid`	0.117	0.105	0.116	0.106
`drv_pos`	0.010	0.032	0.017	0.032
`drv_pts_share`	0.030	0.025	0.021	0.025
`drv_wins`	0.007	0.007	0.016	0.007
Base value	0.140	0.142	0.141	0.142

Reading. The **grid is the most important input for every model**, at least three times any other input. Blue dots (front slots) sit on the right: a front slot raises the probability, a slot further back lowers it. The standings columns share the rest, and each model splits them differently: logistic regression leans on `drv_pts_share`, the forest on `drv_pos`, and the network spreads it over all three. They carry much the same information (VIF above), so how the credit is shared says more about the model than about Formula 1. For the forest, the two explainers agree: the averages match to 0.0001, and the largest difference in a single value is 0.009.

12.2 Local: SHAP force plots and LIME

Two validation rows are picked by a rule on the data, not on any model, and explained for all three models: the *hit* is the first validation race where the championship leader started from pole and finished on the podium; the *miss* is the podium finisher who started furthest back. LIME [7] perturbs the row 5,000 times, asks the model about each copy, and fits a small linear model around the row.

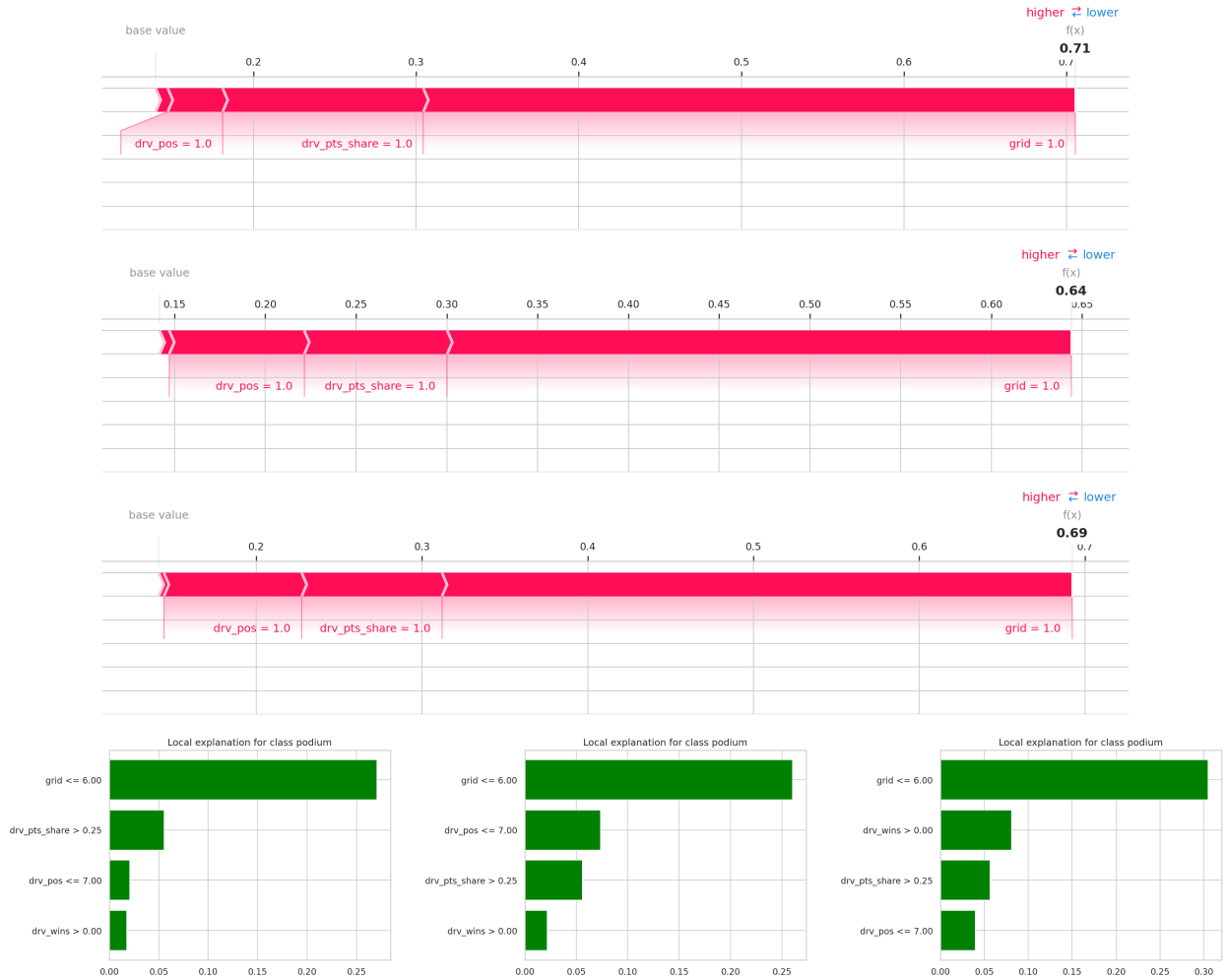


Figure 22: Hit: Sebastian Vettel, 2018 Bahrain GP. Top: SHAP force plots for logistic regression, forest and network (red pushes up, blue down, from the base value to $f(x)$). Bottom: LIME for the same three (green = towards podium, red = against).



Figure 23: Miss: Sebastian Vettel, 2019 German GP. Same layout as Figure 22.

Table 18: SHAP values and LIME weights for the forest, and each model's probability.

	Hit: Vettel, Bahrain 2018			Miss: Vettel, Germany 2019		
Input	Value	SHAP	LIME	Value	SHAP	LIME
`grid`	1	+0.344	+0.260	20	−0.105	−0.114
`drv_pos`	1	+0.074	+0.074	4	+0.033	+0.062
`drv_pts_share`	1.00	+0.079	+0.056	0.552	+0.024	+0.048
`drv_wins`	1	+0.005	+0.022	0	−0.004	−0.026
P(podium) LR / RF / NN	0.71 / 0.64 / 0.69			0.01 / 0.09 / 0.04		
LIME local fit R^2 (3 models)	0.80–0.83			0.22–0.27		

Reading. *Hit:* the championship leader on pole; every model says podium, above its threshold. For the forest, from the average of about 0.14, `grid = 1` adds the most, then the points share and the position; one win adds almost nothing. The other models also put the grid first. *Miss:* Vettel started 20th and finished on the podium; every model says no. `grid = 20` pulls the probability down (−0.11 to −0.14 across the models), while the driver's standings (4th, 55% of the leader's points) push it up a little. Given what is known before the start a low probability is right: only 1.6% of starts from 11th or further back end on the podium (Q2). The race was wet (outside knowledge), which nothing before the start can show. SHAP and LIME point the same way for every input, row and model. LIME's local fit is good for the hit and poor for the miss, so for the miss the SHAP values are more trustworthy. SHAP splits the difference from the average exactly

among the inputs; LIME's weights come from an approximate local model and need not add up.

13 Inference functions

`PREDICT[model]` is one inference function per model. Each takes one driver-race as a dictionary of pre-race features in their normal units, fills missing values with the training medians through the same imputer and scaler as in training, and returns the probability and the yes/no at that model's validation threshold. A key that is not a feature (a typo) is rejected instead of being treated as missing *(notebook §17)*.

Table 19: Inference on a complete record, a partially-missing record and a real test row with missing standings (P(podium), yes if above the model's threshold).

Record	Baseline	LR	RF	NN
Complete: Verstappen, Saudi Arabian GP 2024 (grid 1, leader, 1 win); finished on the podium	0.591 yes	0.705 yes	0.644 yes	0.692 yes
Partially missing: only `grid = 4` (a rookie in round 1; standings filled with 12, 0.076, 0)	0.348 no	0.240 no	0.338 no	0.271 no
Real test row: Leclerc, Bahrain GP 2022 (round 1, standings missing); finished on the podium	0.591 yes	0.416 yes	0.477 yes	0.504 yes
Thresholds	0.427	0.399	0.432	0.348

14 Limitations

- **Qualifying-coverage drift.** Qualifying times exist only from 1994 (Q2 from 2005, Q3 from 2006, and only for the faster drivers). The models use the grid and no qualifying times, so information such as the gap to pole is not used.
- **Era sensitivity.** Field sizes, points systems and reliability changed a lot over 75 years. Points are a share of the leader's points and the split is by season, but the 1950s training rows still differ strongly from 2022–2024, and the test seasons are a new set of car rules.
- **Dominant teams.** Mercedes took 45% of validation podiums and Red Bull 37% of test podiums, so the scores and explanations describe those eras.
- **Race-day events are invisible.** Rain, crashes, safety cars and failures are not known before the start; the miss row is an example.
- **Small validation and test sets** (243 and 204 podiums): differences of about 0.01 PR-AUC between models could turn around with other seasons.
- **One validation block does several jobs** (form window, feature groups, settings, early stopping, thresholds), so validation scores are a little optimistic.
- **Missing standings are filled as a typical driver** (the training median); a round-1 favourite looks like a mid-table driver. No constructors' standings exist before 1958, and the 1950–1960 standings include Indy 500 drivers (33–44 per season), which pushes other drivers' positions down.
- **Feature selection adds whole groups in a fixed order,** and only grid, field size, standings and form were candidates. Sprint results are not used.
- **Q1–Q3** rely on our own grouping of 137 statuses and a nationality-to-country mapping; renamed teams are kept apart.
- **Explanations are not causal** and describe these models on 2018–2021 rows.

15 Conclusion

After auditing and cleaning the 14 tables, a leakage-safe table of 24,933 entries was built and split by season. Front-row starts convert to podiums far more often since 2014 (55.5% → 76.0%) and the circuit ranking reshuffled completely; there is no home advantage once the starting position is held fixed; and mechanical retirements fell from 11.2% to 6.9% of starts. For prediction, the grid plus the driver's standings before the race are enough: logistic regression, a random forest and a shallow network all clearly beat the grid-only baseline (test PR-AUC 0.706–0.720 against 0.647) and are about equally good. Ablation and SHAP agree that the grid carries most of the signal and the season standings add a smaller but consistent part.

References

[1] Alan Agresti and Brent A. Coull. Approximate is better than "exact" for interval estimation of binomial proportions. *The American Statistician*, 52(2):119–126, 1998.

[2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[3] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.

[4] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 4765–4774, 2017.

[5] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[6] Rohan Rao. Formula 1 world championship (1950–2024). Kaggle dataset, `https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020`, 2024. Based on the Ergast Developer API.

[7] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "Why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.

[8] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

A Reproducibility and methods

The notebook runs top to bottom with *Run All*. On Kaggle the CSV folder is found automatically and the first cell installs `shap`, `lime` and `missingno`; the versions used (Python 3.13, numpy 2.1.3, pandas 2.3.3, scikit-learn 1.6.1, TensorFlow 2.20.0, shap 0.52.0, lime 0.2.0.1) are printed by the second cell and pinned in `environment.txt`. All random seeds are 42.

Table 20: Methods used and where they come from.

Method	Question it answers	From
`read_csv`, `info`/`describe`, missingno	What is in the tables, what is missing	Lab 1
Median imputation, `StandardScaler` fit on train	Fill gaps and scale without leaking	Labs 1, 3
Logistic regression, random forest	Statistical models and baseline	Labs 2, 4
Keras network, early stopping	Shallow neural network	Lab 3
PR-AUC, ROC-AUC, F1, precision, recall	Prediction quality	Lab 2, brief
SHAP (permutation, `TreeExplainer`), LIME	Global and local explanations	Lab 4, brief
Wilson interval	Uncertainty of a rate from few cases	Wilson (1927)
Spearman correlation	Monotonic relations, rankings	rank version of Lab 1
Odds ratio from logistic regression	Home effect at the same start and era	Lab 2 model
VIF	Is a column mostly a copy of the others	Lab 2 linear regression
Race bootstrap	Can two windows / two feature sets be told apart	Efron & Tibshirani (1993)